Although IAM is specific to a single cloud, AWS, its longevity and scale are unmatched. AWS has reported that its identity layer processes over half a billion API calls every second (https://aws.amazon.com/awstv/watch/44e6f1abd16/), resulting in trillions of policy evaluations daily. The popularity of AWS has made IAM the first—and sometimes only—policy language many developers encounter.

8.4.1 How IAM works

IAM policies are written in JSON and specify permissions for principals (users and roles) and AWS resources. AWS evaluates all relevant policies, including identity-based policies, resource-based policies, Service Control Policies (SCPs), session policies, and permission boundaries, before deciding whether to allow or deny a request. Deny rules override allow rules, and the evaluation order is carefully defined in AWS's documentation.

IAM policies are specific to AWS.

- Actions correspond directly to AWS API calls (such as `"s3:GetObject"`).
- Resources are AWS objects identified by Amazon Resource Names (ARNs).
- Context is derived from request metadata, AWS global condition keys, and custom tags.
- Principals include IAM users, roles, or federated identities like those from AWS IAM Identity Center.

Because IAM policies align closely with AWS's architecture, they feel native to anyone building on the platform. This integration enables the language to express fine-grained permissions with minimal ambiguity, leveraging AWS's built-in understanding of its services, identifiers, and context keys. While this makes IAM especially powerful within AWS, it also confines its use to that environment, an acceptable trade-off for the millions of developers who rely on AWS daily.